

Practice Question Set

Core Concepts & Memory Layout

1. What is a string in C, and how is it related to a character array?
2. What is a null character, what is its notation, and what is its ASCII value?
3. How is a zero (0) stored differently from the character '0' in a string array's memory?
4. What happens when you provide a size smaller than the string literal's actual required space during array initialization (e.g., `char str[5] = "hello";`)?
5. What value does an array name evaluate to when passed without an index, and what does `printf("%s", str);` expect?

Input/Output Operations

6. Why is it unnecessary to use the address-of operator (&) when scanning a string using `scanf("%s", s1);`?
7. What are the major limitations of using `scanf()` to read strings from a user?
8. What is the standard purpose of the `gets()` function, and why is its usage considered risky or dangerous?
9. Write the proper syntax of the `fgets()` function and explain how it prevents memory safety issues.
10. What special output behavior does `puts()` display automatically compared to a regular `printf("%s", ...)` statement?

Built-in String Library Functions

11. What is the return type and parameter type of the `strlen()` library function?
12. What does the `strcat()` function do, and what serious problem can occur if the destination buffer is too small?
13. How does `strncat()` differ from the standard `strcat()` function?
14. Explain the three different output states of the `strcmp()` function when comparing two strings.
15. What is the role of `strcpy()`, and which argument is the source and which is the destination?

Important Interview/Viva Questions

16. Why does passing an individual element like `str[i]` to `printf("%c", str[i]);` inside a loop work, whereas passing it to `printf("%s", str[i]);` fails?
17. Explain the risk of "buffer overflow" when using `scanf()` and `gets()`. Which standard function should be used to fix this?
18. What does it mean when we say "string literals are implemented as arrays of characters in C"? How does this affect functions like `strcat()`?
19. How do you find the length of a string without using the built-in `strlen()` library function?
20. How can you modify an individual character inside a string using its ASCII value instead of a character literal?

- 21. Why does a direct comparison like `if (str1 == str2)` fail to check if two strings contain the same text? What should be used instead?**
- 22. What is the technical difference between declaring a function parameter as `char *str` versus `char str[]`?**

Answers

1. What is a string in C, and how is it related to a character array?

Answer: A string is a sequence of characters. Technically, a character array is considered a valid string if and only if it ends with a null character (`'\0'`). The compiler does not inherently distinguish between an array and a string; it depends entirely on how the programmer uses the data.

2. What is a null character, what is its notation, and what is its ASCII value?

Answer: A null character marks the termination point of a string loop. Its notation is written as `'\0'`, and its mathematical ASCII value is exactly 0.

3. How is a zero (0) stored differently from the character '0' in a string array's memory?

Answer: In a character array layout, a plain zero (0) is stored directly as an integer null value (`\0`), which terminates a string loop. Conversely, '0' is stored as a character value which carries its corresponding numerical character meaning rather than a termination marker.

4. What happens when you provide a size smaller than the string literal's actual required space during array initialization (e.g., `char str[5] = "hello";`)?

Answer: The actual size required for "hello" is 6 bytes (5 for the characters + 1 for the null terminator). Specifying a size of 5 overrides or clips the space needed for the null marker. As a result, when a while loop checks for `'\0'`, it fails to hit the terminator, leading to inaccurate or corrupted text outputs.

5. What value does an array name evaluate to when passed without an index, and what does `printf("%s", str);` expect?

Answer: The name of a string array without an index automatically evaluates to its base address (the memory location of the very first character). When printing via `%s`, the statement expects this base address so it can track and stream characters sequentially until a null terminator is hit.

6. Why is it unnecessary to use the address-of operator (`&`) when scanning a string using `scanf("%s", s1);`?

Answer: Because the name of the character array (`s1`) already implicitly acts as a pointer that passes the base memory address of the string to the function. Therefore, adding an explicit `&` operator is technically unnecessary and redundant.

7. What are the major limitations of using `scanf()` to read strings from a user?

Answer:

It is incapable of reading multiword strings because it treats any space, tab, or newline character as an absolute terminating point.

It automatically injects a `'\0'` string terminator whenever it encounters a space or tab.

It fails to check buffer sizes, leaving it highly vulnerable to memory crashes or overflow issues if the user inputs too many characters.

8. What is the standard purpose of the `gets()` function, and why is its usage considered risky or dangerous?

Answer: The `gets()` function is specifically designed to read an entire single line of string input, including spaces. However, it is dangerous because it completely ignores the allocated buffer limit of the destination array while capturing input, exposing the program directly to a **buffer overflow** crash.

9. Write the proper syntax of the `fgets()` function and explain how it prevents memory safety issues.

Answer:

Syntax: `fgets(array_name, array_size, stdin);`

Prevention: It completely stops buffer overflows because it strictly reads a maximum of `(array_size - 1)` characters from the input stream and automatically reserves the final spot `(+1)` to safely write the null terminator `('\\0')`.

10. What special output behavior does puts() display automatically compared to a regular printf("%s", ...) statement?

Answer: Unlike printf(), the puts() function automatically appends a newline operator (\n) to the end of the string output, moving the cursor to the next line automatically.

11. What is the return type and parameter type of the strlen() library function?

Answer:

Return Type: size_t (which represents an unsigned integer).

Parameter Type: char * (a pointer directed to the starting character of a target string).

12. What does the strcat() function do, and what serious problem can occur if the destination buffer is too small?

Answer: The strcat() function handles the concatenation (appending) of a source string onto the end of a destination string. If the destination array does not have a large enough capacity to hold both strings together, the function will overwrite adjacent memory blocks, resulting in a **buffer overflow** error.

13. How does strncat() differ from the standard strcat() function?

Answer: While strcat() copies the entire source string onto the destination string, strncat() restricts the operation by concatenating only a specific, limited portion or number of characters from the source string.

14. Explain the three different output states of the strcmp() function when comparing two strings.

Answer: The function compares strings based on ASCII values and returns:

0: If both strings are completely identical and equal.

Negative value (-ve): If the first non-matching character of the first string is lexicographically smaller than that of the second string.

Positive value (+ve): If the first string is lexicographically greater than the second string.

15. What is the role of strcpy(), and which argument is the source and which is the destination?

Answer:

Role: It is used to copy the complete contents of one string over into another string location.

Syntax usage: strcpy(destination_string, source_string); — it overwrites the first string argument with the characters stored in the second string argument

Important Interview Answers

16. Why does passing an individual element like str[i] to printf("%c", str[i]); inside a loop work, whereas passing it to printf("%s", str[i]); fails?

Answer: When you pass str[i], you are passing a single, specific character value to the function, which %c expects. However, the %s format specifier does not look for a character value; it strictly requires a **memory address** (the base address of a string). Because a single character value is not a valid memory pointer, printf("%s", ...) cannot trace subsequent bytes and will fail or crash the program.

17. Explain the risk of "buffer overflow" when using scanf() and gets(). Which standard function should be used to fix this?

Answer: Both scanf() and gets() are highly risky because they do not verify or enforce the upper boundaries of an array's storage space while collecting input. If a user types more characters than the array can hold, these functions will blindly overwrite neighboring system memory, causing a **buffer overflow** or a fatal system crash. To completely eliminate this threat, programmers should utilize **fgets()**, which strictly caps the input length to match the declared array size.

18. What does it mean when we say "string literals are implemented as arrays of characters in C"? How does this affect functions like strcat()?

Answer: In C, when you type a plain text string literal (like "Gaurav"), the compiler behind the scenes handles it exactly like an array of characters stored in memory. The text name itself automatically functions as a pointer pointing straight to its first letter. Because it yields a pointer, you can pass a raw string literal directly into functions like strcat(str2, "Gaurav");, and the function will successfully locate and read the source data.

19. How do you find the length of a string without using the built-in `strlen()` library function?

Answer: You can determine the length manually by stepping through the character array using a loop (while or for) and tracking a counter variable. The loop condition checks each index position to see if it has encountered the null character (`str[i] != '\0'`). As long as the character at the current index is valid, the counter increments by 1. Once the null character is reached, the loop stops, and the accumulated counter represents the string's length.

20. How can you modify an individual character inside a string using its ASCII value instead of a character literal?

Answer: Characters in C are structurally backed by integer ASCII codes. Because of this, you can target a specific element using its index and assign a raw integer value directly to it. For example, writing `str[1] = 97;` updates the character at index 1 to a lowercase 'a', because 97 is the predefined ASCII representation for that specific letter.

21. Why does a direct comparison like `if (str1 == str2)` fail to check if two strings contain the same text? What should be used instead?

Answer: Using the equality operator (`==`) on two string variables merely compares their **base memory addresses** (the locations where the two arrays start in RAM) rather than analyzing the individual letters inside them. Since two separate arrays will always occupy completely distinct slots in memory, the comparison will return false even if their textual contents match perfectly. To properly assess the text, you must use **`strcmp()`** or build a loop that manually tests each corresponding element index (`str1[i] == str2[i]`).

22. What is the technical difference between declaring a function parameter as `char *str` versus `char str[]`?

Answer: Technically, inside a function prototype or definition header, both notations are completely identical to the C compiler; both explicitly signal that the function expects the **memory address of a character pointer**. However, writing

`char str[]` acts as an excellent self-documenting convention for programmers, making it visually obvious that the function is looking for a structured collection of characters (an array) rather than a lone, single character address.